render_md_to_pdf.sh and check_pdf_coverage.py

Field	Value
Scripts	`scripts/docs/render_md_to_pdf.sh`, `scripts/docs/check_pdf_coverage.py`, `scripts/docs/print.css`, `scripts/docs/print_landscape.css`, `scripts/docs/pdf_table_cols.py`
Purpose	Render the `.pdf` twin of a Markdown doc without clipping wide tables or long tokens, and prove it by comparing the words of the Markdown with the text extracted from the PDF (constitution 11.4.65, 11.4.201)
Why	PDFs rendered from the pandoc HTML with default print rules were cut off at the right page edge (up to 29% of a doc's words missing); the file opened fine, so nothing noticed
Render	`render_md_to_pdf.sh FILE.md ...` writes `FILE.pdf` (pandoc gfm to HTML5 with `--metadata document-css=false`, because pandoc's own `<style>` (body `max-width: 36em`) outranks the print sheets in WeasyPrint; `pdf_table_cols.py` adds computed table column widths; then WeasyPrint with `print.css`). The PDF's Keywords field carries `src-sha256:<hex>` of the Markdown
Orientation rule	A doc whose Markdown has a table separator row with 5 or more columns is rendered A4 landscape (`print_landscape.css`), all others A4 portrait
Print rules	Body 9.5pt at full page width, fixed table layout at 100% width with column widths from `pdf_table_cols.py` (widths in points: every column gets the estimated glyph width of its longest word, capped at 24 characters and shrinking to 16 when the columns do not fit, plus padding, so a short id such as B100 or a `file:line` number never wraps; the rest by content weight), 7.5pt table text, words break only when over-long (`overflow-wrap: break-word`), rows never split across pages, `pre` and `code` wrap
`--check`	`render_md_to_pdf.sh --check FILE.md` writes nothing and runs the coverage check on the existing PDF
Coverage check	`check_pdf_coverage.py [--root DIR] [--allow FILE] [--min-len N] [--quiet] FILE.md ...`
Method	Distinct tokens of 4 or more characters (a token containing a digit from 2 characters) of the Markdown, Unicode letters included (Cyrillic, CJK) (link targets, HTML comments and fence markers removed, NFKC) must appear in `pdftotext` output on a word boundary and contiguously: a line wrap is tolerated only after a separator character (`-/._:@=&?`) or inside Markdown words of 16 or more characters, and in both cases only when the next piece starts on a later line and not right of the previous piece's end, so a lost table cell cannot be matched by gluing neighbouring cells together. Two reading orders (default and `-raw`)
Allow-list	`scripts/docs/pdf_coverage_allow.tsv` (`path`, `token`, `reason`; the reason is required). Default: no file, nothing allowed
Control	PDF side: a token taken from the PDF must be found and a fabricated one not. Markdown side: the matcher must find a Markdown token in the Markdown's own text and a fabricated one not. Run for every file of a multi-file call; a blank PDF whose Markdown has tokens is FAIL, not OK. Otherwise BLIND
Exit	0 no token missing; 1 tokens missing, PDF absent, empty or unreadable; 2 BLIND, usage or tool missing
Tests	`scripts/docs/tests/test_check_pdf_coverage.py` (clipped long code line, body width and font, landscape threshold, long cell URL, lost-cell repro, garbage and stale PDF, modified-doc scope, link rewrite; mutations: no table CSS, truncated PDF, broken needles, deleted twin)

Reproducible: the render sets `SOURCE_DATE_EPOCH` and a PDF identifier derived from the Markdown, so two renders are byte-identical (checked on the shipped PDFs). The twin gate also covers untracked new docs under `docs/`.

Limits: a doc with no tokens of 4 or more characters (or digits) reports OK with nothing to compare. Glyph widths are class-based estimates, not measured font metrics. A column too narrow for a long word breaks it mid-word as a last resort (the text stays present and the check allows it only inside words of 16 or more characters). The `src-sha256` Keywords value is metadata and can be forged; the twin gate's coverage run catches a forged old-content PDF only when the new Markdown adds tokens the old PDF lacks. This proves the words are present in the extracted text, not that the page looks right; a token split by a page break between two table cells can read as missing and is a reason to fix the layout, not to allow-list it. Tokens shorter than 4 characters without a digit are not measured.

check_twin_presence.py

Field	Value
Script	`scripts/docs/check_twin_presence.py`
Purpose	Gate: every Markdown doc added or modified under `docs/` since a ref has current `.html` and `.pdf` twins (constitution 11.4.65)
Usage	`check_twin_presence.py [--root DIR] [--since REF] [--no-html-drift] [--quiet]`; `--since` defaults to the newest tag
Scope	`.md` files added or modified (`git diff --diff-filter=AM REF..HEAD`, plus working-tree changes) under `docs/`; a modified doc is in scope when it already has a twin of either kind, an added doc always is

Field	Value
Checks	`.html` non-empty and equal to `render_md_to_html.sh --check` output; `.pdf` non-empty, `%PDF`, readable (`pdfinfo`), its Keywords `src-sha256` equals the sha256 of the current Markdown (stale PDF fails), and `check_pdf_coverage.py` reports 0 missing (`--no-pdf-coverage`, `--no-html-drift` skip those)
Control	A doc with both twins must read complete and one missing its pdf incomplete; an empty scope is BLIND
Exit	0 all present, 1 one or more `MISSING` rows, 2 BLIND or usage
Fix	`render_md_to_html.sh FILE.md` and `render_md_to_pdf.sh FILE.md`, never hand-edit generated twins